Due Date: 3/19/2026 Names: Aiden Wang

CSCI 240 PA3 Submission

Exercise 1

Source code below:

```
package PA3;
import java.util.ArrayList;

class MyArrayList extends ArrayList<String> {

   public void removeElem(String str) {
       if (this.contains(str)) {
           this.remove(str);
       }
   }
}
public class PA3_Ex1 {
   public static void main(String[] args) {
       System.out.println("Modified by: Aiden Wang\n");
       MyArrayList vector = new MyArrayList();
       vector.add(0, "Two");
       vector.add(1, "Three");
       vector.add(0, "One");
       vector.add(3, "Four");
       System.out.println("After additions: " + vector);
       vector.remove(0);
       vector.removeElem("Two");
       vector.removeElem("four");
       vector.add(1, "Aiden");
       System.out.println("After removals and inserting name: " +
vector);
   }
}
```

Input/output below:

```
/Users/guoqingwang/Library/Java/JavaVirtualMachines/temurin-22
Modified by: Aiden Wang

After additions: [One, Two, Three, Four]
After removals and inserting name: [Three, Aiden, Four]

Process finished with exit code 0
```

Due Date: 3/19/2026 Names: Aiden Wang

Exercise 2

Source code below:

```java
package PA3;

import net.datastructures.LinkedPositionalList;
import net.datastructures.Position;
import java.util.Iterator;

public class PA3_Ex2 {
   public static void main(String[] args) {
       System.out.println("Modified by: Aiden Wang\n");

       LinkedPositionalList<String> list = new LinkedPositionalList<>();

       list.addFirst("Two");
       list.addLast("Three");
       list.addFirst("One");
       list.addLast("Four");
       System.out.print("List contents: ");
       Iterator<String> it = list.iterator();
       while (it.hasNext()) {
           System.out.print(it.next() + " ");
       }
       System.out.println();
       Position<String> cursor = list.first();
       cursor = list.after(cursor);
       cursor = list.after(cursor);
       list.addBefore(cursor, "Aiden");
       list.remove(list.first());
       list.remove(list.last());
       System.out.print("After modifications: ");
       it = list.iterator();
       while (it.hasNext()) {
           System.out.print(it.next() + " ");
       }
       System.out.println();
   }
}
```

Input/output below:

```
/Users/guoqingwang/Library/Java/JavaVirtualMach
Modified by: Aiden Wang

List contents: One Two Three Four
After modifications: Two Aiden Three

Process finished with exit code 0
```

Due Date: 3/19/2026 Names: Aiden Wang

Exercise 3
Source code below:

```java
package PA3;

import net.datastructures.LinkedPositionalList;
import net.datastructures.Position;

public class PA3_Ex3 {

   public static class TextEditor {
       private LinkedPositionalList<Character> list;
       private Position<Character> cursor;
       public TextEditor(String s) {
           list = new LinkedPositionalList<>();
           for (char c : s.toCharArray()) {
               if (Character.isLetter(c) || c == ' ') {
                   list.addLast(c);
               }
           }
           cursor = null;
       }
       public void display() {
           Position<Character> walk = list.first();
           while (walk != null) {
               if (walk == cursor) {
                   System.out.print("|");
               }
               System.out.print(walk.getElement());
               walk = list.after(walk);
           }
           if (cursor == null) {
               System.out.print("|");
           }
           System.out.println();
       }
       public void left() {
           if (cursor == null) {
               cursor = list.last();
           } else if (cursor != list.first()) {
               cursor = list.before(cursor);
           }
       }
       public void right() {
           if (cursor != null) {
               cursor = list.after(cursor);
           }
       }
       public void first() {
           cursor = list.first();
       }

       public void last() {
           cursor = null;
       }
```

Due Date: 3/19/2026 Names: Aiden Wang

```
    public void insert(char c) {
        if (Character.isLetter(c) || c == ' ') {
            if (cursor == null) {
                list.addLast(c);
            } else {
                list.addBefore(cursor, c);
            }
        }
    }
    public void remove() {
        if (list.isEmpty()) return;
        if (cursor == null) {
            list.remove(list.last());
        } else if (cursor != list.first()) {
            list.remove(list.before(cursor));
        }
    }
}
public static void main(String[] args) {
    System.out.println("Modified by: Aiden Wang\n");

    TextEditor editor = new TextEditor("HHello Word");

    editor.display();
    editor.right();
    editor.left();
    editor.insert('1');
    editor.insert('1');
    editor.display();
    editor.first();
    editor.right();
    editor.remove();
    editor.last();
    editor.display();
}
}
```

Input/output below:

```
/Users/guoqingwang/Library/Java/JavaVirtualMachines
Modified by: Aiden Wang

HHello Word|
HHello Worl|d
Hello World|

Process finished with exit code 0
```

Due Date: 3/19/2026 Names: Aiden Wang

Question 1:
Would you use an array to implement List ADT? Explain why or why not.

Answer for Q1:

Yes – usually the best default

Pros

- O(1) random access (get/set by index)
- Excellent cache performance (contiguous memory)
- Very low memory overhead
- O(1) amortized append at end

Cons

- O(n) insert/delete in middle or beginning (must shift elements)

Use array-based List when: random access, iteration, sorting, or general use is common.
Use linked list instead only if: frequent inserts/deletes at beginning or middle + almost no random access.

Default rule: Use array (dynamic/resizable) for ~95% of List cases.

Question 2:
Given a choice between a vector and a list, when should you select a vector over a list?

Answer for Q2:

Pick Vector / ArrayList (almost always)

Why Vector wins most cases:

- O(1) random access → LinkedList is O(n)
- Much faster iteration & cache performance
- Lower memory use (no per-node pointers)
- Faster sorting & searching

Use LinkedList only when:

- Frequent add/remove at **beginning** or **middle**
- You already have iterator positioned
- Random access is rare / not needed

One-line rule (exam/interview gold):

- Start with **ArrayList / vector** → switch to **LinkedList** only if you clearly need frequent middle/beginning modifications and don't need fast indexing.

Due Date: 3/19/2026 Names: Aiden Wang

Extra Credit
Source code below:

```
package PA3;

public class PA3_EC {

   public static class TextEditor1 {
       private StringBuilder text;
       private int cursor;

       public TextEditor1(String s) {
           text = new StringBuilder();
           for (char c : s.toCharArray()) {
               if (Character.isLetter(c) || c == ' ') {
                   text.append(c);
               }
           }
           cursor = text.length();        }

       public void display() {
           StringBuilder out = new StringBuilder(text);
           out.insert(cursor, "|");
           System.out.println(out.toString());
       }

       public void left() {
           if (cursor > 0) cursor--;
       }

       public void right() {
           if (cursor < text.length()) cursor++;
       }

       public void first() {
           cursor = 0;
       }

       public void last() {
           cursor = text.length();
       }

       public void insert(char c) {
           if (Character.isLetter(c) || c == ' ') {
               text.insert(cursor, c);
               cursor++;            }
       }

       public void remove() {
           if (cursor > 0) {
               text.deleteCharAt(cursor - 1);
               cursor--;
           }
       }
   }

   public static void main(String[] args) {
```

```
        System.out.println("Modified by: Aiden Wang\n");
        TextEditor1 editor = new TextEditor1("HHello Word");
        editor.display();
        editor.right();
        editor.left();
        editor.insert('1');
        editor.insert('l');
        editor.display();
        editor.first();
        editor.right();
        editor.remove();
        editor.last();
        editor.display();
    }
}
```

Input/output below: